When Migrations Go Bad

Migrations suffer from one serious problem. The underlying DDL statements that update the database schema are not transactional. This isn't a failing in Rails—most databases don't support the rolling back of `create table`, `alter table`, and other DDL statements.

Let's look at a migration that tries to add two tables to a database:

```
class ExampleMigration < ActiveRecord::Migration
  def change
    create_table :one do ...
    end
    create_table :two do ...
    end
  end
end
```

In the normal course of events, the `up` method adds tables, `one` and `two`, and the `down` method removes them.

But what happens if there's a problem creating the second table? We'll end up with a database containing table `one` but not table `two`. We can fix whatever the problem is in the migration, but now we can't apply it—if we try, it will fail because table `one` already exists.

We could try to roll the migration back, but that won't work. Because the original migration failed, the schema version in the database wasn't updated, so Rails won't try to roll it back.

At this point, you could mess around and manually change the schema information and drop table `one`. But it probably isn't worth it. Our recommendation in these circumstances is simply to drop the entire database, re-create it, and apply migrations to bring it back up-to-date. You'll have lost nothing, and you'll know you have a consistent schema.

All this discussion suggests that migrations are dangerous to use on production